

Web Services

Parte 1 - Introduzione

Prof. **Marco Camurri**

Argomenti

- Il problema della interoperabilità
- Il concetto di Remote Procedure Call
- Web service
- Web API
- Invocazione di ws in Java (la classe URL)
- Invocazione di ws in JavaScript con AJAX
- Operazioni CRUD
- Web service di tipo REST
- Web service basati su SOAP e WSDL

Interoperabilità

- E' frequente la necessità di far comunicare tra loro sistemi software diversi (prodotti da aziende diverse e con tecnologie diverse)
- **Esempio 1:** il sistema informatico della banca X deve comunicare con il sistema della banca Y ogni volta che vi è un trasferimento di denaro tra conti delle due banche
- **Esempio 2:** un software per la gestione robotizzata di un magazzino deve comunicare con il software gestionale dell'azienda, per aggiornare le informazioni sulla disponibilità dei prodotti ogni volta che viene effettuato un prelievo di articoli dal magazzino
- **Esempio 3:** il sistema informativo di un centro prenotazioni presso un areoporto deve comunicare con il sistema informativo centrale di una compagnia aerea per effettuare la prenotazione di un volo

Interoperabilità

- Problema: i due sistemi sono stati scritti da softwarehouse diverse, usano linguaggi diversi, girano su sistemi operativi diversi, ecc..

Il problema dell'**interoperabilità del software** consiste nel far comunicare fra loro sistemi software eterogenei, a causa di:

- linguaggi di programmazione diversi
- S.O diversi
- architetture hardware diverse

Interoperabilità

In generale non è possibile trasferire dati direttamente in formato binario tra software eterogenei, poichè diversi linguaggi di programmazione implementano i tipi di dati in modo diverso.

Esempio 1

- una variabile intera potrebbe occupare 2 byte in un linguaggio di programmazione, e 4 byte in un altro linguaggio
- quindi il software A non può semplicemente inviare il contenuto della variabile in binario su una connessione TCP e aspettarsi che il software B lo legga correttamente
- lo stesso vale, a maggior ragione, per i tipi di dato più complessi come stringhe, numeri non interi, array, oggetti, ...
- ... le stringhe del C sono sequenze in cui ogni carattere occupa un byte, e sono terminate dal carattere '\0' , ma ciò non è vero in altri linguaggi

Interoperabilità

- Un'altro problema di cui tener conto quando si trasmettono dati in binario dall'**ordinamento dei byte (Big Endian / Little endian)**
- CPU differenti utilizzano diverse convenzioni per l'ordinamento dei byte all'interno di una parola
- Esempio:
 - CPU con registri a 32 bit
 - Come viene inviato sulla rete il numero
11111111 01010101 11110000 00000000 ?
 - Se si usa l'ordinamento little-endian, il primo byte inviato sarà 00000000 (byte meno significativo)
 - Se si usa l'ordinamento big-endian, il primo byte inviato sarà 11111111 (byte più significativo)

Interoperabilità

I problemi precedenti si possono risolvere inviando i dati in formato testo (es. ASCII). Anche in questo caso però occorre tenere in conto di alcune differenze:

Esempio 3

- Sistemi operativi diversi utilizzano caratteri diversi per indicare la fine di una riga in un file di testo
- Su LINUX il terminatore di riga è il carattere `'\n'` (line feed)
- Windows utilizza invece la sequenza di due caratteri `'\r'` e `'\n'` (carriage return + line feed)
- Quindi possono esserci problemi se il software A (su LINUX) invia stringhe terminate da `'\n'` al software B (su Windows) e quest'ultimo le trasferisce direttamente su un file di testo senza modificare il terminatore

Remote Procedure Call

- Per far comunicare due software eterogenei (A e B) sarebbe utile disporre di un sistema standard che consenta di invocare, dal codice dell'applicazione A, una funzione dell'applicazione B, come se si trattasse di una chiamata locale ...
- ... anche se in realtà la funzione è eseguita nell'applicazione B, che si trova su un computer remoto della rete
- Esempio: funzione che recupera da un server le informazioni su un prodotto a partire dal codice del prodotto

```
Prodotto p = getInformazioniProdotto("1534"); // RPC  
int quantita = p.getQtaMagazzino();  
String descrizione = p.getDescrizione();
```


Remote Procedure Call

- La possibilità appena descritta è indicata in informatica con il termine **Remote Procedure Call**

Remote Procedure Call = invocazione da parte di un programma di una funzione (o metodo) attivata su un computer diverso da quello su cui il programma è in esecuzione

Metodi remoti, operazioni, servizi

- Nella pratica la chiamata remota è realizzata tramite l'invio di una richiesta ad un server
- il server estrae i parametri della richiesta, legge/modifica dati in un database, e produce una risposta (che può contenere i dati richiesti, o l'esito dell'operazione)
- Altri esempi di chiamate remote

```
int codiceConferma = PrenotaVolo(numeroVolo, numeroPosti);  
String numeroOrdine = confermaOrdineAcquisto(carrello);
```

- Ogni **metodo** corrisponde ad un **servizio** offerto dal server
- Si dice anche il server **espone** un insieme di **operazioni** al client

Remote Procedure Call

- la sintassi della chiamata remota dovrebbe risultare simile a quella di una chiamata locale, senza che il programmatore debba preoccuparsi dei "dettagli" legati a:
 - conversione dei parametri in un formato condiviso tra chiamante e chiamato, adatto al trasporto attraverso la rete (**marshalling**)
 - invio dei parametri attraverso la rete
 - attesa e ricezione del valore di ritorno, riconversione del valore di ritorno nel formato del chiamante (**unmarshalling**)

Esempio

- Per rendere semplice la comunicazione tra sistemi software realizzati da aziende diverse, occorre accordarsi sull'utilizzo di **standard semplici e condivisi** per la realizzazione di chiamate remote (vedi slide successiva)
- **Esempio:** una compagnia aerea ha interesse a rendere disponibile in rete un servizio che fornisca informazioni sui propri voli, facilmente richiamabile da altre applicazioni
- **Motivo:** se è facile interagire con il sistema informativo della compagnia aerea, sarà più facile, da parte di una diversa softwarehouse, creare una nuova applicazione che ricerca e confronta soluzioni di viaggio da varie compagnie aeree

Tecnologie per la realizzazione di RPC

- Nel tempo sono state proposte numerose tecnologie (non prive di difetti) per la realizzazione di chiamate a procedura remote nei linguaggi di programmazione:
 - **Java RMI (Remote Method Invocation)**
consente di invocare un metodo su una JVM remota.
Limitazione: le due applicazioni comunicanti devono essere scritte entrambe in Java.
 - **DCOM (Distributed Component Object Model)**
Soluzione proposta da Microsoft, solo per Windows (ora superata da .NET)
 - **CORBA**
 - ...
 - Sistemi basati su **web services** (soluzione definitiva???)

Cos'è un web service?

- Un **web service** (**ws**) è un sistema software che offre un servizio rivolto ad un altro sistema software, utilizzando gli stessi protocolli e tecnologie utilizzati dal World Wide Web (HTTP e HTTPS)
- Il servizio offerto spesso consiste nel consentire l'accesso (controllato) ad una sorgente di dati (es. database) attraverso la rete
- Al contrario di **web applications** o **web sites**, le risposte dei ws sono progettate per essere utilizzate da un altro software, anziché da un utente umano
- Per questo motivo, mentre un web site restituisce tipicamente pagine **HTML**, un ws restituisce i dati usando formati facilmente interpretabili da un programma:
- I formati più comuni sono **XML** e **JSON**

Definizione di web-service

- Riassunto, per essere classificato come web service, un server deve possedere due requisiti fondamentali:
 - 1) offrire un servizio pensato per essere fruibile da un programma (cioè non rivolto direttamente ad un utente). Ciò implica la restituzione dei dati in formati come XML e JSON, ma non HTML!
 - 2) utilizzare protocolli standard del web come mezzo di comunicazione (HTTP, HTTPS)
- Il punto 2) è essenziale nella definizione di ws:

NON è classificabile come web service un server che accetta richieste su una socket TCP utilizzando un proprio protocollo (non standard) direttamente al di sopra di TCP!

Esempio di web service: geocoding

- I servizi di **geocoding** e **geocoding inverso** forniti da Google sono un esempio di web service
- **Geocoding** = processo di conversione da un indirizzo testuale (strada Spolverina, Mantova) a coordinate geografiche (latitudine e longitudine)
- **Reverse Geocoding** = processo inverso di conversione da coordinate geografiche a indirizzo testuale

<https://developers.google.com/maps/documentation/geocoding/intro>

Esempio di web service: geocoding

Il servizio di geocoding è richiamabile tramite un'URL così formata:

https://maps.googleapis.com/maps/api/geocode/*outputFormat*?*parameters*

outputFormat deve essere sostituito con:

- **xml** se desideriamo che la risposta sia in formato XML
- **json** se desideriamo che la risposta sia in formato JSON

parameters contiene i parametri della richiesta

- **address** indirizzo testuale di cui si vuole effettuare il geocoding
- **key** la prima volta che si usa il servizio è necessario richiedere a e una "API Key", che identifica l'applicazione e che dovrà essere passate anche in tutte le richieste successive. La API Key fornita gratuitamente consente un numero limitato di richieste giornaliere.

Esempio di chiamata al servizio di geocoding

- Per chiamare il ws di geocoding di Google è sufficiente scriverne l'URL (con i parametri richiesti) in un browser:

`https://maps.googleapis.com/maps/api/geocode/xml?
address=strada%20spolverina%20mantova&key=.....`

Esempio di chiamata al servizio di geocoding

https://maps.googleapis.com/maps/api/geocode/xml?address=strada%20spolverina%20mantova&key=AlzaS

parametri

```
<GeocodeResponse>
  <status>OK</status>
  <result>
    <type>route</type>
    <formatted_address>Via Spolverina, 46030 Cerese MN, Italia</formatted_address>
    <address_component>...</address_component>
    <address_component>...</address_component>
    <address_component>...</address_component>
    <address_component>...</address_component>
    <address_component>...</address_component>
    <address_component>...</address_component>
    <geometry>
      <location>
        <lat>45.1279180</lat>
        <lng>10.7792478</lng>
      </location>
      <location_type>GEOMETRIC_CENTER</location_type>
    </geometry>
    <viewport>
      <southwest>
        <lat>45.1238294</lat>
        <lng>10.7717231</lng>
      </southwest>
      <northeast>
        <lat>45.1320174</lat>
        <lng>10.7867729</lng>
      </northeast>
    </viewport>
  </result>
</GeocodeResponse>
```

Parametri nella URL

Se l'url del web service prevede dei parametri dopo il **?**, questi devono seguire le regole standard per le URL, ovvero:

- ogni parametro è una coppia nome=valore
- se vi sono più coppie, si separano con **&**
- nell'intera URL può comparire solo un particolare sottoinsieme di caratteri ASCII. I caratteri non validi devono essere codificati secondo la codifica **URL encoding**

Esempio: spazio (codice ASCII 32) --> %20

Web API

- I termini **web API** e **web service** sono a volte usati come sinonimi...
- In realtà, una web API è l'insieme di specifiche che descrivono come invocare uno o più servizi web (url del servizio, parametri richiesti, formato della risposta, ecc..)
- In altre parole, la web API è l'**interfaccia** del servizio, mentre il web service è il programma che implementa il servizio
- Per richiamare un web service è sufficiente conoscere la web API ..
- ..mentre non è necessario conoscere la tecnologia con cui è stato implementato il web service (pagina .php , server Java, pagina .asp , ecc...)

URL Encoding in Java

Per ottenere la versione "url-encoded" di una stringa in Java:

```
String s = URLEncoder.encode("ciao mondo", "UTF-8");  
// ora s = "ciao%20mondo"
```

Invocazione di ws da codice Java

- La vera utilità dei ws consiste nel poterli richiamare "da codice"
- In Java è possibile richiamare un ws utilizzando la classe **java.net.URL**

```
String s = "https://maps.googleapis.com/.../xml?" + parametri;  
s = URLEncoder.encode(s, "UTF-8");  
URL url = new URL(s);  
InputStream in = url.openStream();
```

- Il metodo **openStream** invia una richiesta (in questo caso una richiesta HTTP) per l'url specificata e restituisce un InputStream per leggere direttamente il contenuto della risposta (saltando quindi gli header della risposta HTTP)

Parsing della risposta XML

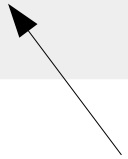
- Se la risposta è in XML è possibile effettuare il parsing con la libreria DOM

```
URL url = new URL("http://.....");
```

```
InputStream in = url.openStream();
```

```
DocumentBuilder docBuilder =  
    DocumentBuilderFactory.newInstance().newDocumentBuilder();
```

```
Document xmlDoc = docBuilder.parse(in);
```



org.w3c.dom.Document

Esercizio

Scrivere un metodo Java che, dato l'indirizzo testuale di un luogo, restituisce le sue coordinate geografiche*

**public static Coordinate getCoordinate(String indirizzo)
throws Exception;**

Nota: Coordinate è una classe così definita:

```
class Coordinate {  
    private double latitudine;  
    private double longitudine;  
    // getter, setter, costruttori  
    ...  
    // toString ...  
}
```

* usando il servizio di geocoding descritto in precedenza

Soluzione

```
public class GeocodeClient {
    public static Coordinate getCoordinate(String indirizzo) throws Exception {
        indirizzo = URLEncoder.encode(indirizzo, "UTF-8");
        String s =
            "https://maps.googleapis.com/maps/api/geocode/xml?address=" +
            indirizzo+ "&key=AIzaSyAhxjUE7-NLVCf8cLZHuqf1-ZOZnUJmOFk;
        URL url = new URL( s );
        InputStream in = url.openStream();
        DocumentBuilder docBuiler
            = DocumentBuilderFactory.newInstance().newDocumentBuilder();
        Document xmlDoc = docBuiler.parse(in);
        Element loc =(Element)xmlDoc.getElementsByTagName("location").item(0);
        Node lat = loc.getElementsByTagName("lat").item(0);
        Node lng = loc.getElementsByTagName("lng").item(0);
        double latitudine = Double.parseDouble( lat.getTextContent() );
        double longitudine = Double.parseDouble( lng.getTextContent() );
        return new Coordinate(latitudine,longitudine);
    }
    public static void main(String[] args) throws Exception {
        System.out.println( getCoordinate("Milano") );
    }
}
```

Formati di risposta di un web-service

- Teoricamente, il contenuto della risposta HTTP prodotta da un ws può essere un file di qualsiasi tipo ...
- ... bisogna però ricordare che la risposta di un ws è destinata ad un software, e non ad un essere umano!
- Il formato della risposta deve essere tale da facilitare l'analisi (**parsing**) della risposta da parte del programma che la riceve
- HTML non è adatto, in quanto i tag html descrivono la struttura del documento (titoli, paragrafi, elenchi, tabelle, ecc..) al fine della sua presentazione grafica
- In pratica, i formati di risposta più utilizzati sono **XML** e **JSON**

Il formato JSON

- JSON = **JavaScript Object Notation**
- È un formato testuale, alternativo a XML, usato spesso per trasferire dati strutturati tramite la rete
- Deriva dalla sintassi di JavaScript per la definizione di oggetti (con alcune limitazioni), ma oggi è utilizzato indipendentemente da JavaScript
- Pur rimanendo autodescrittivi, i documenti in formato JSON sono più compatti rispetto a XML (sintassi più "leggera")
- E' particolarmente utile ricevere una risposta in formato JSON quando il programma chiamante è scritto in JavaScript
- Infatti è immediato costruire un oggetto JavaScript a partire da una stringa JSON tramite la funzione **JSON.parse**:

var mioOggetto = JSON.parse(stringa);

Sintassi JSON

- La sintassi JSON si basa su due costrutti fondamentali:
 - **Oggetto**, creato tramite una coppia di graffe { }
 - **Array**, creato tramite una coppia di quadre []
- Un oggetto è composto da una o più coppie del tipo *key* : *value* separate da virgola
- *key* è sempre una stringa con doppi apici
- *value* può essere:
 - una stringa (solo doppi apici)
 - un numero (senza apici)
 - una delle costanti **true**, **false** o **null** (senza apici)
 - un oggetto
 - un array (elenco di *value* separati da virgola, racchiuso tra [])

Esempio: creazione di un oggetto in JS

```
<script>
  var oggetto =
  {
    "nome" : "Mario" ,
    "cognome" : "Rossi",
    "stipendio" : 1500.00,
    "responsabile": true,
    "data_nascita" : { "giorno": 1, "mese": 2, "anno": 1990} ,
    "e_mails" : [ "mail1@abc.com", "mail2@xyz.it" ] ,
    "titoli" :
      [
        { "tipo": "diploma", "punteggio": 100 } ,
        { "tipo": "laurea", "punteggio": 110 }
      ]
  };

  alert( oggetto.nome );
  alert( oggetto.data_nascita.anno );
  alert( oggetto.titoli[0].punteggio );
</script>
```

Esempio di documento in formato JSON

```
{
  "id_studente": 1315,
  "nome": "Luca",
  "anno": 5,
  "sezione": "A",
  "valutazioni":
    [
      { "materia": "Inglese",
        "data": "2015-01-25",
        "voto": 9
      },
      { "materia": "Matematica",
        "data": "2015-01-27",
        "voto": 6
      }
    ]
}
```

**File con
estensione .j
son**



Differenze fra sintassi JSON e JavaScript

Esistono alcune differenze fra la sintassi JSON e sintassi generale di JavaScript per la creazione di oggetti

Ad esempio:

- In JS è possibile omettere le virgolette nelle chiavi:
var oggetto = { nome: "mario", cognome: "rossi" };
- In JS è possibile usare gli apici singoli
- In JS i valori possono anche essere delle funzioni (metodi)
- Ecc..

Esempio di validatore JSON:

<https://www.freeformatter.com/json-validator.html>

Esempio: parsing di JSON in JavaScript

```
<script>
```

```
    var stringa = '{ "nome" : "mario" , "cognome" : "rossi" }';
```

```
    var oggetto = JSON.parse(stringa);
```

```
    alert( oggetto.cognome );
```

```
</script>
```

Confronto fra JSON e XML

```
{
  "id_studente": 1315,
  "nome": "Luca",
  "anno": 5,
  "sezione": "A",
  "valutazioni":
    [
      { "materia": "Inglese",
        "data": "2015-01-25",
        "voto": 9
      },
      { "materia": "Matematica",
        "data": "2015-01-27",
        "voto": 6
      }
    ]
}
```

```
<?xml version="1.0" ?>
<studente>
  <id>1315</id>
  <nome>Luca</nome>
  <anno>5</anno>
  <sezione>A</sezione>
  <valutazioni>
    <valutazione>
      <materia>Inglese</materia>
      <data>2015-01-25</data>
      <voto>9</voto>
    </valutazione>
    <valutazione>
      <materia>Matematica</materia>
      <data>2015-01-27</data>
      <voto>6</voto>
    </valutazione>
  </valutazioni>
</studente>
```

Esercizio

- Dato il seguente file XML, indica il contenuto di un possibile file in formato JSON contenente le stesse informazioni

```
<?xml version="1.0"?>
<person id="101">
  <firstname>Kenneth</firstname>
  <lastname>Garcia</lastname>
  <address>
    <street>950 Towel Lane</street>
    <city>Foster City</city>
    <state>CA</state>
  </address>
</person>
```

Soluzione

```
{
  "id": 101,
  "firstName": "Kenneth",
  "lastName": "Garcia",
  "address": {
    "street": "950 Tower
Lane",
    "city": "Foster City"
    "state": "CA"
  }
}
```

Chiamate a web service in JavaScript

- In JavaScript è possibile effettuare chiamate a web service grazie ad AJAX
- **AJAX** = **A**synchronous **J**avaScript **A**nd **X**ML
- AJAX non è nuovo linguaggio, nè una libreria
- Il termine AJAX indica semplicemente una tecnica di programmazione JavaScript che si basa su due punti:
 - l'utilizzo di un oggetto predefinito nei browser (l'oggetto **XMLHttpRequest**) che consente al codice javascript di inviare una richiesta HTTP ad un server e aggiornare parti della pagina html in base alla risposta ricevuta
 - L'utilizzo di XML* (e relativo parser DOM) come formato per il trasferimento dei dati da server a codice javascript (o viceversa)

* in realtà oggi è sempre più frequente l'uso di JSON al posto di XML

Chiamate a web service in JavaScript

- Nota: per motivi di sicurezza l'oggetto XMLHttpRequest può effettuare richieste solo a server che si trovano sullo stesso dominio della pagina che contiene il codice JavaScript
- Questa limitazione imposta dai browser è detta **same origin policy**
- Esempio: il codice JS della pagina **www.abc.com/test.html** può contattare un web service all'indirizzo **api.abc.com/test.php** , mentre NON può contattare un server all'indirizzo **www.xyz.com/test.php**
- Per superare questa limitazione il server che fornisce la pagina con il codice JavaScript deve includere speciali intestazioni nel messaggio HTTP...
- Per approfondire si veda CORS (Cross-Origin Requests) e le intestazioni Access-Control-Allow-Origin di HTTP

Motivi del successo dei web services

Rispetto alle altre tecnologie per la realizzazione di chiamate a procedura remota, i web service si sono rivelati una soluzione vincente al problema dell'interoperabilità del software a causa dei seguenti punti di forza:

- Si basano su **standard** noti, aperti, ben documentati, e diffusi su tutte le piattaforme (HTTP/HTTPS, XML, ...)
- Praticamente ogni linguaggio di programmazione dispone di librerie per effettuare richieste HTTP e leggerne la risposta, o per effettuare il parsing di file XML
- Utilizzando HTTP o HTTPS, i messaggi da/verso un web service **possono facilmente attraversare i firewall** (le richieste sono destinate alla porta 80 !)
- Le richieste e le risposte utilizzano formati testuali (spesso autodescrittivi come XML o JSON) evitando quindi la necessità di accordarsi sul formato binario dei dati (è sufficiente utilizzare la stessa codifica dei caratteri, es "UTF-8")
- Sono facili da implementare (è sufficiente un qualsiasi linguaggio di programmazione web lato server) e facili da testare (non è necessario scrivere un client: per testare il servizio è sufficiente un browser)

Operazioni CRUD

Cosa significa la frase:

"il web service supporta (o espone) le operazioni CRUD " ?

- CRUD = **C**reate, **R**ead (o **R**etrieve), **U**ppdate, **D**eleate
- Sono le 4 operazioni effettuabili su un generico database:
 - Create = **creazione** di un nuovo elemento/record
 - Read = **lettura** di un elemento/record
 - Update = **modifica** di un elemento/record esistente
 - Delete = **cancellazione** di un elemento/record

Operazioni CRUD

Se il database è un DB relazionale, allora:

- creare un elemento significa creare un nuovo record
- le operazioni CRUD corrispondono alle seguenti istruzioni SQL:
 - **Create** --> **INSERT**
 - **Read** --> **SELECT**
 - **Update** --> **UPDATE**
 - **Delete** --> **DELETE**

Operazioni CRUD

L'utilizzo tipico di un web service è quello di consentire ad applicazioni esterne l'accesso controllato ad un database

- accesso controllato = non tutte le modifiche al db sono consentite, ma solo quelle previste dal ws!
- Es: compagnia aerea espone un ws per effettuare ricerche sul db dei voli ed effettuare prenotazioni, ma non per inserire nuovi voli !
- un ws fornisce (espone) le operazioni CRUD se, per una o più tabelle, consente aggiunta, lettura, aggiornamento e modifica di record
- Es: aggiunta di una prenotazione, visualizzazione delle prenotazioni effettuate, modifica di una prenotazione, cancellazione di una prenotazione